Online Photography Portfolio

Web Technologies Lab · Source Code (Week 10 to Week 14)

Week 10	Write an Ajax / Fetch program to send data using the POST method.
Week 11	Develop TypeScript programs using classes, interfaces and functions.
Week 12	Create Angular components and implement data binding, directives.
Week 13-14	Develop Angular forms, routing, and services for a sample single-page application.

Week 10 · Send Data using POST

Write an Ajax / Fetch program to send data using the POST method.

week_10/ajax.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Send Data - Fetch POST</title>
    <link rel="stylesheet" href="../week_3/style.css">
</head>

<body>

<h1>Contact Photographer</h1>

<nav>
    <a href="../week_1/home.html">Home</a>
    <a href="../week_5/catalogue.html">Catalogue</a>
    <a href="../week_9/ajax.html">Photographer Data</a>
    <a href="ajax.html">Contact</a>
</nav>

<div class="form-container">

    <h2>Feedback Form</h2>

    <label>Name:</label>
    <input type="text" id="name" required>

    <label>Email:</label>
    <input type="email" id="email" required>

    <label>Message:</label>
    <textarea id="message" required></textarea>

    <button type="button" onclick="sendData()">Send</button>

</div>

<pre id="result"></pre>

<script src="post.js"></script>

</body>
</html>
```

week_10/post.js

```
function sendData() {

    const payload = {
        name: document.getElementById("name").value,
        email: document.getElementById("email").value,
        message: document.getElementById("message").value
    };

    if (!payload.name || !payload.email || !payload.message) {
        document.getElementById("result").innerText =
            "Please fill all the fields before sending.";
        return;
    }
```

```
    fetch("https://jsonplaceholder.typicode.com/posts", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify(payload)
    })

        .then(response => {

            if (!response.ok) {
                throw new Error("Network error: " + response.status);
            }

            return response.json();
        })

        .then(data => {
            document.getElementById("result").innerText =
                "Server Response:\n" + JSON.stringify(data, null, 2);
        })

        .catch(error => {
            document.getElementById("result").innerText =
                "Error: " + error.message;
        });
}
```

Week 11 · TypeScript Program

Develop TypeScript programs using classes, interfaces and functions.

Compile with **npx tsc** inside week_11 to produce portfolio.js.

week_11/typescript.html

```
<!DOCTYPE html>
<html>
<head>
    <title>TypeScript Demo</title>
    <link rel="stylesheet" href="../week_3/style.css">
</head>

<body>

<h1>Photographer Portfolio - TypeScript</h1>

<nav>
    <a href="../week_1/home.html">Home</a>
    <a href="../week_5/catalogue.html">Catalogue</a>
    <a href="../week_9/ajax.html">Photographer Data</a>
    <a href="../week_10/ajax.html">Contact</a>
    <a href="typescript.html">TypeScript</a>
    <a href="../week_12/output/index.html">Angular Gallery</a>
    <a href="../week_13_14/output/index.html">Book a Shoot</a>
</nav>

<div class="ajax-container">

    <h2>Week 11 Demo</h2>

    <p>Written in <b>portfolio.ts</b> using an interface, a class and a function.</p>

    <button type="button" onclick="showPortfolio()">Show Portfolio</button>

</div>

<pre id="result"></pre>

<script src="portfolio.js"></script>

</body>
</html>
```

week_11/portfolio.ts

```
interface Photo {
    title: string;
    price: number;
}
class Photographer {
    constructor(public name: string, public photos: Photo[]) { }
    total(): number {
        return this.photos.reduce((sum, photo) => sum + photo.price, 0);
    }
}
const ayesha = new Photographer("Ayesha Khan", [
    { title: "Snow Leopard", price: 5000 },
    { title: "Misty Valley", price: 3500 }
]);

function showPortfolio(): void {
    let text = ayesha.name + "\n\n";
```

```
    ayesha.photos.forEach(photo => {
        text = text + photo.title + " - Rs. " + photo.price + "\n";
    });
    document.getElementById("result")!.innerText = text + "\nTotal: Rs. " + ayesha.total();
}
```

week_11/portfolio.js

```
"use strict";
class Photographer {
    constructor(name, photos) {
        this.name = name;
        this.photos = photos;
    }
    total() {
        return this.photos.reduce((sum, photo) => sum + photo.price, 0);
    }
}
const ayesha = new Photographer("Ayesha Khan", [
    { title: "Snow Leopard", price: 5000 },
    { title: "Misty Valley", price: 3500 }
]);
function showPortfolio() {
    let text = ayesha.name + "\n\n";
    ayesha.photos.forEach(photo => {
        text = text + photo.title + " - Rs. " + photo.price + "\n";
    });
    document.getElementById("result").innerText = text + "\nTotal: Rs. " + ayesha.total();
}
```

week_11/tsconfig.json

```
{
    "compilerOptions": {
        "target": "ES2015",
        "module": "es2015",
        "strict": true,
        "skipLibCheck": true,
        "outDir": "."
    },
    "files": ["portfolio.ts"]
}
```

Week 12 · Angular Components

Create Angular components and implement data binding, directives.

Run with **npm install** then **npx ng serve** inside week_12, or **npx ng build** to create week_12/output/ which is linked from the Home page.

week_12/src/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Week 12 - Angular Components</title>
  <base href="./">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

week_12/src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

week_12/src/styles.css

```
/* Reuse the same stylesheet as the earlier weeks */
@import "../../week_3/style.css";

.cheap {
  color: #16a34a;
  font-weight: bold;
}

.costly {
  color: #d97706;
  font-weight: bold;
}
```

week_12/src/app/app.config.ts

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners()
  ]
};
```

week_12/src/app/photo.ts

```
// The shape of one photo object
export interface Photo {
  title: string;
  category: string;
  price: number;
```

```
  image: string;
}
```

week_12/src/app/app.component.ts

```
import { Component } from '@angular/core';
import { NgFor, NgIf } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Photo } from './photo';
import { PhotoCardComponent } from './photo-card/photo-card.component';

@Component({
  selector: 'app-root',
  imports: [NgFor, NgIf, FormsModule, PhotoCardComponent],
  templateUrl: './app.component.html'
})
export class AppComponent {
  title = 'Photo Gallery - Angular';
  photographer = 'Ayesha Khan';
  searchText = '';
  showPrices = true;
  likes = 0;
  lastLiked = '';

  photos: Photo[] = [
    { title: 'Mountain', category: 'Nature', price: 5000,
      image: 'https://images.unsplash.com/photo-1506905925346-21bda4d32df4?w=600' },
    { title: 'Wildlife', category: 'Wildlife', price: 3500,
      image: 'https://images.unsplash.com/photo-1474511320723-9a56873867b5?w=600' },
    { title: 'Portrait', category: 'Portrait', price: 2500,
      image: 'https://images.unsplash.com/photo-1494790108377-be9c29b29330?w=600' },
    { title: 'Travel', category: 'Travel', price: 4000,
      image: 'https://images.unsplash.com/photo-1488646953014-85cb44e25828?w=600' }
  ];

  // Returns only the photos whose category matches the search box
  filteredPhotos(): Photo[] {
    return this.photos.filter(photo =>
      photo.category.toLowerCase().includes(this.searchText.toLowerCase())
    );
  }

  togglePrices(): void {
    this.showPrices = !this.showPrices;
  }

  // Runs when a child PhotoCardComponent sends the "liked" event
  onLiked(photoTitle: string): void {
    this.likes++;
    this.lastLiked = photoTitle;
  }
}
```

week_12/src/app/app.component.html

```
<!-- Interpolation: {{ }} -->
<h1>{{ title }}</h1>

<nav>
  <a href="../../week_1/home.html">Home</a>
  <a href="../../week_5/catalogue.html">Catalogue</a>
  <a href="../../week_11/typescript.html">TypeScript</a>
  <a href="index.html">Angular Gallery</a>
  <a href="../../week_13_14/output/index.html">Book a Shoot</a>
```

```
</nav>

<div class="ajax-container">
  <h2>Week 12 Demo</h2>
  <p>Photographer: <b>{{ photographer }}</b></p>

  <!-- Two-way binding: [(ngModel)] -->
  <input type="text" [(ngModel)]="searchText" placeholder="Search category">
  <p>You typed: {{ searchText }}</p>

  <!-- Event binding: (click) -->
  <button (click)="togglePrices()">
    {{ showPrices ? 'Hide Prices' : 'Show Prices' }}
  </button>

  <p>Total likes: {{ likes }}</p>
  <!-- *ngIf shows this line only after a photo is liked -->
  <p *ngIf="lastLiked">Last liked: {{ lastLiked }}</p>
</div>

<div class="photo-container">
  <!-- *ngFor repeats the child component for every photo -->
  <!-- Property binding [photo] sends data DOWN, event (liked) sends data UP -->
  <app-photo-card
    *ngFor="let p of filteredPhotos()"
    [photo]="p"
    [showPrice]="showPrices"
    (liked)="onLiked($event)">
  </app-photo-card>
</div>

<p *ngIf="filteredPhotos().length === 0">No photos found.</p>
```

week_12/src/app/photo-card/photo-card.component.ts

```
import { Component, EventEmitter, Input, Output } from '@angular/core';
import { NgClass, NgIf, NgStyle } from '@angular/common';
import { Photo } from '../photo';
import { HighlightDirective } from '../highlight.directive';

@Component({
  selector: 'app-photo-card',
  imports: [NgIf, NgClass, NgStyle, HighlightDirective],
  templateUrl: './photo-card.component.html'
})
export class PhotoCardComponent {
  // Data coming IN from the parent
  @Input() photo!: Photo;
  @Input() showPrice = true;

  // Event going OUT to the parent
  @Output() liked = new EventEmitter<string>();

  likePhoto(): void {
    this.liked.emit(this.photo.title);
  }
}
```

week_12/src/app/photo-card/photo-card.component.html

```
<!-- appHighlight is our own custom attribute directive -->
<div class="photo-card" appHighlight>

  <!-- Property binding: [src] and [alt] -->
```

```
  <img class="photo-thumb" [src]="photo.image" [alt]="photo.title">

  <h3>{{ photo.title }}</h3>
  <p>{{ photo.category }} Photography</p>

  <!-- *ngIf + ngClass: price is green when cheap, orange when costly -->
  <p *ngIf="showPrice" [ngClass]="photo.price > 3000 ? 'costly' : 'cheap'">
    Rs. {{ photo.price }}
  </p>

  <!-- ngStyle sets a style from a value -->
  <p [ngStyle]="{ 'font-size': '13px', 'color': '#64748b' }">
    Hover over me!
  </p>

  <button (click)="likePhoto()">Like</button>
</div>
```

week_12/src/app/highlight.directive.ts

```
import { Directive, ElementRef, HostListener } from '@angular/core';

// Custom attribute directive: changes the background on mouse hover
@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  constructor(private el: ElementRef) { }

  @HostListener('mouseenter') onMouseEnter() {
    this.el.nativeElement.style.backgroundColor = '#fef3c7';
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.el.nativeElement.style.backgroundColor = 'white';
  }
}
```

Week 13-14 · Angular Forms, Routing and Services

Develop Angular forms, routing, and services for a sample single-page application.

Run with **npm install** then **npx ng serve** inside week_13_14, or **npx ng build** to create week_13_14/output/ which is linked from the Home page.

week_13_14/src/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Week 13-14 - Angular SPA</title>
  <base href="./">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

week_13_14/src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

week_13_14/src/styles.css

```
/* Reuse the same stylesheet as the earlier weeks */
@import "../../week_3/style.css";

nav a.active {
  color: #f59e0b;
}

.error {
  display: block;
  color: #dc2626;
  font-size: 13px;
}

/* Angular adds these classes to form fields automatically */
input.ng-invalid.ng-touched,
select.ng-invalid.ng-touched {
  border-color: #dc2626;
}

button:disabled {
  background: #94a3b8;
  cursor: not-allowed;
}
```

week_13_14/src/app/models.ts

```
// Shapes of the data used in the app
export interface Photo {
  id: number;
  title: string;
```

```
  category: string;
  price: number;
  image: string;
}

export interface Booking {
  name: string;
  email: string;
  category: string;
  date: string;
  message: string;
}
```

week_13_14/src/app/app.config.ts

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter, withHashLocation } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    // Hash URLs (#/gallery) work with Live Server without extra setup
    provideRouter(routes, withHashLocation())
  ]
};
```

week_13_14/src/app/app.routes.ts

```
import { Routes } from '@angular/router';
import { HomeComponent } from './pages/home/home.component';
import { GalleryComponent } from './pages/gallery/gallery.component';
import { BookingComponent } from './pages/booking/booking.component';
import { BookingsComponent } from './pages/bookings/bookings.component';

// Each path shows a different component
export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'gallery', component: GalleryComponent },
  { path: 'book', component: BookingComponent },
  { path: 'bookings', component: BookingsComponent },
  { path: '**', redirectTo: '' }
];
```

week_13_14/src/app/app.component.ts

```
import { Component } from '@angular/core';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  templateUrl: './app.component.html'
})
export class AppComponent {
  title = 'Photo Booking - Angular SPA';
}
```

week_13_14/src/app/app.component.html

```
<h1>{{ title }}</h1>

<nav>
```

```
  <a href="../../week_1/home.html">Portfolio Home</a>
  <a href="../../week_12/output/index.html">Angular Gallery</a>
  <!-- routerLink changes the page without reloading -->
  <a routerLink="/" routerLinkActive="active" [routerLinkActiveOptions]="{ exact: true }">Home</a>
  <a routerLink="/gallery" routerLinkActive="active">Gallery</a>
  <a routerLink="/book" routerLinkActive="active">Book a Shoot</a>
  <a routerLink="/bookings" routerLinkActive="active">My Bookings</a>
</nav>

<!-- The component for the current route is shown here -->
<router-outlet></router-outlet>
```

week_13_14/src/app/services/photo.service.ts

```
import { Injectable } from '@angular/core';
import { Photo } from '../models';

// A service keeps data in one place so any component can use it
@Injectable({
  providedIn: 'root'
})
export class PhotoService {
  private photos: Photo[] = [
    { id: 1, title: 'Mountain', category: 'Nature', price: 5000,
      image: 'https://images.unsplash.com/photo-1506905925346-21bda4d32df4?w=600' },
    { id: 2, title: 'Wildlife', category: 'Wildlife', price: 3500,
      image: 'https://images.unsplash.com/photo-1474511320723-9a56873867b5?w=600' },
    { id: 3, title: 'Portrait', category: 'Portrait', price: 2500,
      image: 'https://images.unsplash.com/photo-1494790108377-be9c29b29330?w=600' },
    { id: 4, title: 'Travel', category: 'Travel', price: 4000,
      image: 'https://images.unsplash.com/photo-1488646953014-85cb44e25828?w=600' }
  ];

  getPhotos(): Photo[] {
    return this.photos;
  }

  getCategories(): string[] {
    return this.photos.map(photo => photo.category);
  }
}
```

week_13_14/src/app/services/booking.service.ts

```
import { Injectable } from '@angular/core';
import { Booking } from '../models';

// Stores the bookings so the Booking page and My Bookings page share them
@Injectable({
  providedIn: 'root'
})
export class BookingService {
  private bookings: Booking[] = [];

  addBooking(booking: Booking): void {
    this.bookings.push(booking);
  }

  getBookings(): Booking[] {
    return this.bookings;
  }
}
```

week_13_14/src/app/pages/home/home.component.ts

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-home',
  imports: [RouterLink],
  templateUrl: './home.component.html'
})
export class HomeComponent { }
```

week_13_14/src/app/pages/home/home.component.html

```
<div class="ajax-container">
  <h2>Welcome</h2>
  <p>This is a single-page application made with Angular.</p>
  <p>Browse the gallery, book a photo shoot and see your bookings.</p>
  <button routerLink="/gallery">View Gallery</button>
  <button routerLink="/book">Book a Shoot</button>
</div>
```

week_13_14/src/app/pages/gallery/gallery.component.ts

```
import { Component } from '@angular/core';
import { NgFor } from '@angular/common';
import { RouterLink } from '@angular/router';
import { Photo } from '../../models';
import { PhotoService } from '../../services/photo.service';

@Component({
  selector: 'app-gallery',
  imports: [NgFor, RouterLink],
  templateUrl: './gallery.component.html'
})
export class GalleryComponent {
  photos: Photo[];

  // Angular gives us the PhotoService (dependency injection)
  constructor(private photoService: PhotoService) {
    this.photos = this.photoService.getPhotos();
  }
}
```

week_13_14/src/app/pages/gallery/gallery.component.html

```
<h2>Gallery</h2>

<div class="photo-container">
  <div class="photo-card" *ngFor="let photo of photos">
    <img class="photo-thumb" [src]="photo.image" [alt]="photo.title">
    <h3>{{ photo.title }}</h3>
    <p>{{ photo.category }} - Rs. {{ photo.price }}</p>
    <!-- Sends the category to the booking page in the URL -->
    <button routerLink="/book" [queryParams]="{ category: photo.category }">Book</button>
  </div>
</div>
```

week_13_14/src/app/pages/booking/booking.component.ts

```
import { Component } from '@angular/core';
import { NgFor, NgIf } from '@angular/common';
import { FormsModule, NgForm } from '@angular/forms';
import { ActivatedRoute, Router } from '@angular/router';
```

```
import { Booking } from '../../models';
import { PhotoService } from '../../services/photo.service';
import { BookingService } from '../../services/booking.service';

@Component({
  selector: 'app-booking',
  imports: [FormsModule, NgFor, NgIf],
  templateUrl: './booking.component.html'
})
export class BookingComponent {
  categories: string[];

  // The form fields are bound to this object with ngModel
  booking: Booking = {
    name: '',
    email: '',
    category: '',
    date: '',
    message: ''
  };

  constructor(
    private photoService: PhotoService,
    private bookingService: BookingService,
    private route: ActivatedRoute,
    private router: Router
  ) {
    this.categories = this.photoService.getCategories();

    // If we came from the Gallery, pre-select that category
    const category = this.route.snapshot.queryParamMap.get('category');
    if (category) {
      this.booking.category = category;
    }
  }

  onSubmit(form: NgForm): void {
    if (form.invalid) {
      return;
    }
    this.bookingService.addBooking({ ...this.booking });
    this.router.navigate(['/bookings']);
  }
}
```

week_13_14/src/app/pages/booking/booking.component.html

```
<div class="form-container">
  <h2>Book a Photo Shoot</h2>

  <!-- Template-driven form: #bookingForm holds the whole form -->
  <form #bookingForm="ngForm" (ngSubmit)="onSubmit(bookingForm)">

    <label>Name:</label>
    <input type="text" name="name" [(ngModel)]="booking.name"
           #name="ngModel" required minlength="3">
    <small class="error" *ngIf="name.invalid && name.touched">
      Name is required (at least 3 letters).
    </small>

    <label>Email:</label>
    <input type="email" name="email" [(ngModel)]="booking.email"
           #email="ngModel" required email>
    <small class="error" *ngIf="email.invalid && email.touched">
      Please enter a valid email.
```

```
    </small>

    <label>Category:</label>
    <select name="category" [(ngModel)]="booking.category" #category="ngModel" required>
      <option value="">-- Select --</option>
      <option *ngFor="let c of categories" [value]="c">{{ c }}</option>
    </select>
    <small class="error" *ngIf="category.invalid && category.touched">
      Please choose a category.
    </small>

    <label>Date:</label>
    <input type="date" name="date" [(ngModel)]="booking.date" #date="ngModel" required>
    <small class="error" *ngIf="date.invalid && date.touched">
      Please pick a date.
    </small>

    <label>Message:</label>
    <textarea name="message" [(ngModel)]="booking.message"></textarea>

    <!-- Button stays disabled until every rule passes -->
    <button type="submit" [disabled]="bookingForm.invalid">Book Now</button>
  </form>
</div>
```

week_13_14/src/app/pages/bookings/bookings.component.ts

```
import { Component } from '@angular/core';
import { NgFor, NgIf } from '@angular/common';
import { RouterLink } from '@angular/router';
import { Booking } from '../../models';
import { BookingService } from '../../services/booking.service';

@Component({
  selector: 'app-bookings',
  imports: [NgFor, NgIf, RouterLink],
  templateUrl: './bookings.component.html'
})
export class BookingsComponent {
  bookings: Booking[];

  constructor(private bookingService: BookingService) {
    this.bookings = this.bookingService.getBookings();
  }
}
```

week_13_14/src/app/pages/bookings/bookings.component.html

```
<h2>My Bookings</h2>

<p *ngIf="bookings.length === 0">
  No bookings yet. <a routerLink="/book">Book a shoot</a>
</p>

<div class="photographer-container">
  <div class="photographer-card" *ngFor="let b of bookings">
    <h3>{{ b.name }}</h3>
    <p>{{ b.email }}</p>
    <p><b>{{ b.category }}</b> on {{ b.date }}</p>
    <p *ngIf="b.message">"{{ b.message }}"</p>
  </div>
</div>
```